

# ComicCraft: AI Comic Story Creator using Gemini Models

|                      |                                                         |
|----------------------|---------------------------------------------------------|
| <b>Date</b>          | 03 October 2026                                         |
| <b>Team ID</b>       | SWTID-2026-6945                                         |
| <b>Project Name</b>  | ComicCraft - AI Comic Story Creator using Gemini Models |
| <b>Maximum Marks</b> | Project Documentation                                   |

|                     |                     |
|---------------------|---------------------|
| <b>Team Size</b>    | 3                   |
| <b>Team Leader</b>  | Tanishka K          |
| <b>Team Members</b> | Monica R, Yazhene S |

## Project Description

**ComicCraft** is a web-based application that uses AI to generate personalised comic stories and illustrations from user prompts. Built with FastAPI and integrated with Google's Gemini models and Stable Diffusion, it automates storylines, dialogue and comic-style imagery. Users give a story prompt, main character name, setting, tone and art style; ComicCraft returns a panel-by-panel story with illustrations, shown in the browser and downloadable as a PDF.

It offers both an HTML interface and JSON API endpoints, and uses Hugging Face Diffusers for image generation, so that every comic feels cohesive and visually engaging even for non-artists.

## Scenarios

**Scenario 1:** A user enters "A brave fox exploring an enchanted forest", names the hero Rusty, and chooses forest, dramatic and anime. ComicCraft uses Gemini Flash for a 5-panel outline, Gemini Pro for narration and dialogue and Stable Diffusion for the artwork, producing a personalised comic.

**Scenario 2:** A user wants something light-hearted. On the form they select *funny* tone and *comic book* art style. The whole pipeline (outline, story and images) runs again with prompts tailored to a humorous narrative and a classic comic-book look, so users can iterate until the mood matches their vision.

**Scenario 3:** After reviewing the preview the user downloads the comic. `layout_builder.py` assembles each panel's title, image and text; `exporters.py` compiles them with FPDF into a timestamped PDF; the user then lands on the export success page.

## Technical Architecture

ComicCraft has three parts: a **frontend** (HTML, CSS, Jinja2) that collects input and shows comics, a **backend** (FastAPI) that routes requests and orchestrates the workflow, and **AI integration** (Gemini via google-genai, Stable Diffusion via Diffusers). See 3. Project Design Phase/Solution Architecture.pdf for the diagram.

## Pre-requisites

- FastAPI knowledge - <https://fastapi.tiangolo.com>
- Uvicorn ASGI server - <https://www.uvicorn.org>
- Python proficiency - <https://docs.python.org/3/>
- HTML, CSS and Jinja2 - <https://www.w3schools.com> and <https://jinja.palletsprojects.com>
- Google Gemini API - <https://ai.google.dev>

- Hugging Face Diffusers and Stable Diffusion - <https://huggingface.co/docs/diffusers>
- PyTorch basics - <https://pytorch.org/docs/stable/index.html>
- PDF generation with FPDF2 - <https://py-pdf.github.io/fpdf2/>
- Git - <https://git-scm.com/docs>
- Python environments (venv, pip) - <https://docs.python.org/3/library/venv.html>

## Project Workflow

| Milestone                           | Activities                                                                      |
|-------------------------------------|---------------------------------------------------------------------------------|
| 1. Model selection and architecture | 1.1 Research and select models; 1.2 Define architecture; 1.3 Set up environment |
| 2. Core functionalities development | 2.1 Develop core functions; 2.2 Implement FastAPI backend                       |
| 3. routes.py development            | 3.1 Main application logic in routes.py                                         |
| 4. Frontend development             | 4.1 Design and build UI; 4.2 Dynamic Jinja2 templates                           |
| 5. Deployment                       | 5.1 Prepare for local deployment; 5.2 Test and verify                           |

## Milestone 1: Model Selection and Architecture

### Activity 1.1: Research and Select the Appropriate Generative AI Model

ComicCraft must produce (a) a structured comic outline, (b) narration and dialogue and (c) an illustration per panel. We compared Gemini Flash (fast structured output), Gemini Pro (detailed creative text) and Stable Diffusion (image generation) on response quality, prompt adherence, speed, stability and documentation.

| Role                   | Model chosen                      | Reason                                                    |
|------------------------|-----------------------------------|-----------------------------------------------------------|
| Panel outline          | Gemini Flash                      | Fast, follows JSON structure reliably, low cost           |
| Narration and dialogue | Gemini Pro                        | Richer creative writing and consistent tone               |
| Illustrations          | Stable Diffusion v1.5 (Diffusers) | Open model, controllable styles, strong community support |

**Note on model names:** the original brief lists models/gemini-1.5-flash, models/gemini-1.5-pro and runwayml/stable-diffusion-v1-5. Google has retired Gemini 1.5 and the SD 1.5 weights moved to stable-diffusion-v1-5/stable-diffusion-v1-5, so the code reads GEMINI\_FLASH\_MODEL, GEMINI\_PRO\_MODEL and SD\_MODEL\_ID from .env (defaults: gemini-flash-latest, gemini-pro-latest, stable-diffusion-v1-5/stable-diffusion-v1-5). The roles are the same as in the brief. Models actually used and tested: **gemini-2.5-flash** for the outline and **gemini-3.5-flash** for the story (gemini-2.5-pro returned 404 and gemini-3.1-pro had no free quota on our API key).

### Activity 1.2: Define the Architecture of the Application

- **Frontend:** index.html (input), comic\_preview.html (result), export\_success.html (confirmation).
- **Backend:** FastAPI routes in routes.py: form and JSON generation, PDF download, export success, image testing.
- **AI integration:** Gemini Flash creates the 5-panel outline; Gemini Pro writes the story; Stable Diffusion draws the panels; results flow to templates and PDF export.

### Activity 1.3: Set Up the Development Environment

```
python -m venv comiccraft-env
comiccraft-env\Scripts\activate           # Windows
pip install -r requirements.txt           # fastapi uvicorn jinja2 python-multipart google-genai diffusers transformers fpdf
uvicorn app.main:app --reload              # http://127.0.0.1:8000 and /docs
```

Project structure:

```
app/ main.py routes.py config.py schemas.py services/{gemini_flash,gemini_pro,image_generator,layout_builder,exports}
templates/ index.html comic_preview.html export_success.html
static/ img/ panels/ exports/ tests/ scripts/ requirements.txt .env.example
```

## Milestone 2: Core Functionalities Development

### Activity 2.1: Develop the Core Functionalities

| Function             | File                        | What it does                                                                                                                               |
|----------------------|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| generate_outline()   | services/gemini_flash.py    | Asks Gemini Flash for a JSON array of 5 panels (panel_number, title, scene_description, image_prompt); cleans and validates the JSON.      |
| generate_story()     | services/gemini_pro.py      | Asks Gemini Pro to turn the outline into caption and narration/dialogue for every panel; returns one formatted text block.                 |
| generate_image()     | services/image_generator.py | Runs Stable Diffusion with the panel prompt plus art-style keywords, sanitises the file name, saves to static/panels and returns the path. |
| build_comic_layout() | services/layout_builder.py  | Parses the story text and matches each panel with its image: list of dicts with panel number, image path and text.                         |

| Function   | File                  | What it does                                                                                               |
|------------|-----------------------|------------------------------------------------------------------------------------------------------------|
| save_pdf() | services/exporters.py | Builds a multi-page PDF with FPDF2 (cover + one page per panel) in static/exports with a timestamped name. |

### Example: outline generation (services/gemini\_flash.py)

```
def generate_outline(story_prompt, character_name, setting, tone, art_style) -> list[dict]:
    prompt = OUTLINE_PROMPT.format(n=config.NUM_PANELS, story_prompt=story_prompt, ...)
    text = call_gemini(config.GEMINI_FLASH_MODEL, prompt, json_mode=True)
    return normalize_outline(_extract_json(text))    # always exactly 5 complete panels
```

### Activity 2.2: Implement the FastAPI Backend

All routing is in routes.py. The HTML form is read with Form(...) parameters and the JSON API uses the PromptRequest Pydantic schema. Both call the same create\_comic() function (outline, story, images, layout, PDF).

```
class PromptRequest(BaseModel):
    story_prompt: str = Field(..., min_length=5, max_length=500)
    character_name: str = Field("Hero", min_length=1, max_length=40)
    setting: str; tone: str; art_style: str
```

## Milestone 3: routes.py Development

### Activity 3.1: Writing the Main Application Logic in routes.py

| Route                | Method | Purpose                                                            |
|----------------------|--------|--------------------------------------------------------------------|
| /                    | GET    | Homepage with the story form (index.html)                          |
| /generate            | POST   | Form submission: run the AI workflow and render comic_preview.html |
| /generate-comic/json | POST   | JSON API: same workflow, returns layout data and the PDF path      |
| /download/{filename} | GET    | Serve the exported PDF (file name checked against comic_*.pdf)     |
| /export-success      | GET    | Confirmation page after the download                               |
| /test-image          | GET    | Developer utility: test image generation from a prompt             |
| /health              | GET    | Liveness check                                                     |

Error handling: invalid input returns HTTP 422 with a readable message; AI failures raise GenerationError and return 502; unexpected errors return 500 without exposing a stack trace. Handlers are plain *def* functions so FastAPI runs the slow AI work in a thread pool without blocking the server.

```
@router.post("/generate", response_class=HTMLResponse)
def generate(request: Request, story_prompt: str = Form(...), character_name: str = Form("Hero"), ...):
    req = PromptRequest(**form)                # validation
    layout, pdf_path = create_comic(req)        # outline -> story -> images -> layout -> PDF
    return templates.TemplateResponse(request, "comic_preview.html", {"layout": layout, "pdf_file": ..., "req": req})
```

## Milestone 4: Frontend Development

### Activity 4.1: Designing and Developing the User Interface

The interface uses HTML, embedded CSS and Jinja2. index.html has labelled fields for story prompt, character name, setting (school, forest, space, city), tone (light-hearted, dramatic, poetic, funny) and art style (anime, pixel art, comic book, realistic) on a scenic background. Buttons have hover effects, panels use rounded corners and shadows, and the layout is centred and responsive.

### Activity 4.2: Creating Dynamic Templates with FastAPI's Jinja2

```
templates = Jinja2Templates(directory="templates")
{% for panel in layout %} <h2>Panel {{ panel.panel_number }}: {{ panel.title }}</h2>
   <p class="scene">{{ panel.scene_description }}</p> ... {% endfor %}
```

Form field names match the Form(...) parameters in FastAPI; comic\_preview.html loops over the layout list; export\_success.html receives the PDF file name.

## Milestone 5: Deployment

### Activity 5.1: Preparing the Application for Local Deployment

Create a virtual environment, install requirements.txt and store secrets in a .env file loaded with python-dotenv:

```
GEMINI_API_KEY=your_gemini_api_key_here
HF_API_KEY=your_huggingface_api_key_here
```

### Activity 5.2: Testing and Verifying Local Deployment

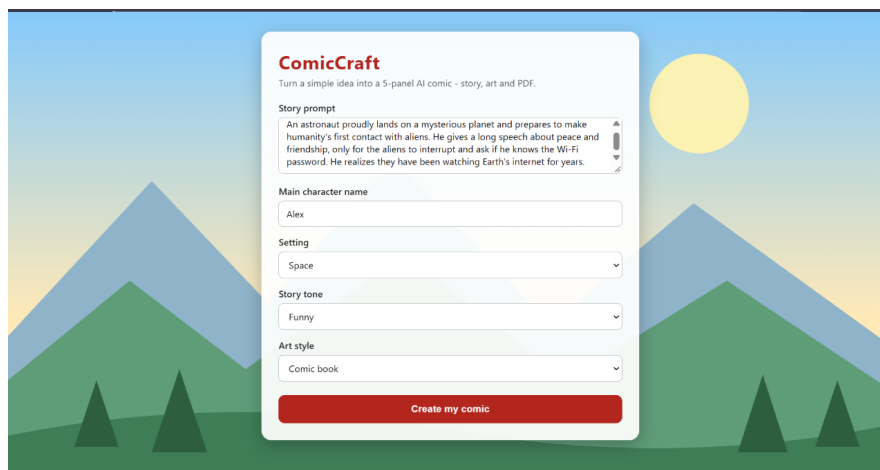
```
uvicorn app.main:app --reload          # open http://127.0.0.1:8000 and http://127.0.0.1:8000/docs
```

Verification performed: 21 automated tests pass (services and routes), four load-test scenarios ran without errors in offline mock mode (COMICCRAFT MOCK=1), and the team completed a full end-to-end run with real Gemini and Stable Diffusion on 30 Sep 2026. The screenshots below were captured from the running application with real Gemini text and Stable Diffusion images.

## Exploring the Website's Web Pages

### Home page

The starting point: a clean form on a scenic background where users enter the story prompt, main character name, setting, tone and art style.



**ComicCraft**  
Turn a simple idea into a 5-panel AI comic - story, art and PDF.

**Story prompt**  
An astronaut proudly lands on a mysterious planet and prepares to make humanity's first contact with aliens. He gives a long speech about peace and friendship, only for the aliens to interrupt and ask if he knows the Wi-Fi password. He realizes they have been watching Earth's internet for years.

**Main character name**  
Alex

**Setting**  
Space

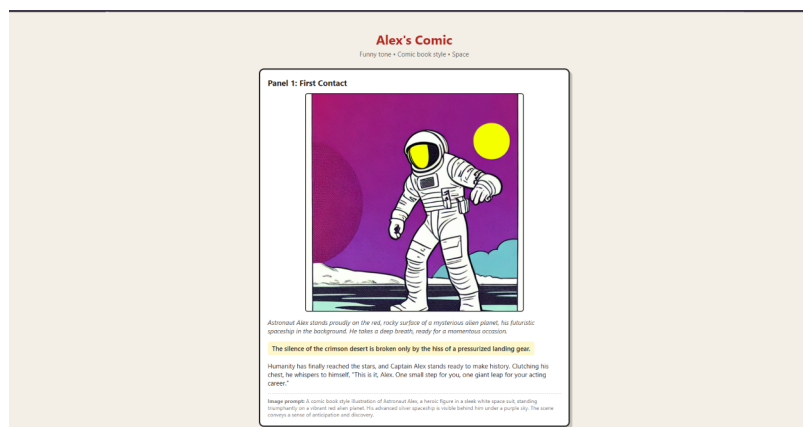
**Story tone**  
Funny

**Art style**  
Comic book

**Create my comic**

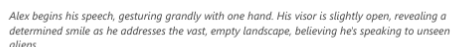
### Comic Preview (result) page

Each panel shows a numbered title, the illustration, the scene description in italics, the caption, the narration and the image prompt used. (Real output from the running application.)



### More panels from the same comic

Panels 2 to 5 of the same comic (funny tone, comic book style, space setting). Each panel shows the generated image, scene description, caption, narration with dialogue, and the image prompt. The "Download Your Comic as PDF" button at the bottom exports the comic as a timestamped PDF.



Clearing his throat, Alex projects his voice to the empty hills with theatrical flair. "People of this glorious world, I bring you greetings from Earth, a beacon of peace, wisdom, and advanced civilization!"

**Image prompt:** A comic book panel showing Astronaut Alex, mid-speech, with a wide, confident grin visible through his partially open helmet visor. He's gesturing dramatically towards the alien landscape, a barren but colorful terrain, conveying a sense of immortality and hope.

After the download starts, users see a confirmation and a "Go Create Another Comic" button.



Your AI-powered comic has been created and downloaded as a PDF. Save it, print it or share it with friends.

File: comic\_20261001\_114442\_461474.pdf

**Download again**

**Go Create Another Comic**

| Date        | Sprint   | Work completed                                                                                                                                                                                                                                                                                                                                                                                                                        | Story points | Cumulative progress  |
|-------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|----------------------|
| 30 Sep 2026 | Sprint-1 | Environment set up; Gemini Flash/Pro and Stable Diffusion compared and selected; config, request schema, story input form and validation built. Real-model checks done: Stable Diffusion downloaded and generating images, Gemini model access tested (gemini-2.5-pro not available on the free key, so gemini-2.5-flash for the outline and gemini-3.5-flash for the story were chosen) and a first full comic generated end to end. | 10           | 10 / 55 points (18%) |
| 01 Oct 2026 | Sprint-2 | Outline service (Gemini Flash) and story service (Gemini) with JSON cleaning and retries; Stable Diffusion image service with placeholder fallback.                                                                                                                                                                                                                                                                                   | 20           | 30 / 55 points (55%) |
| 02 Oct 2026 | Sprint-3 | Layout builder, timestamped PDF export, comic preview, download and export success pages, JSON API and /test-image route.                                                                                                                                                                                                                                                                                                             | 16           | 46 / 55 points (84%) |

| Date        | Sprint   | Work completed                                                                              | Story points | Cumulative progress   |
|-------------|----------|---------------------------------------------------------------------------------------------|--------------|-----------------------|
| 03 Oct 2026 | Sprint-4 | 21 automated tests, performance test, full documentation, demo video and GitHub submission. | 9            | 55 / 55 points (100%) |

## Conclusion

ComicCraft shows how large language models (Gemini Flash and Pro) and Stable Diffusion can work together to turn a simple prompt into a complete illustrated comic with an on-screen preview and a downloadable PDF. Its modular design (one module per workflow step, a shared pipeline and configurable models) makes future features easy to add, such as user accounts, a comic library, character consistency, voice narration and cloud deployment.